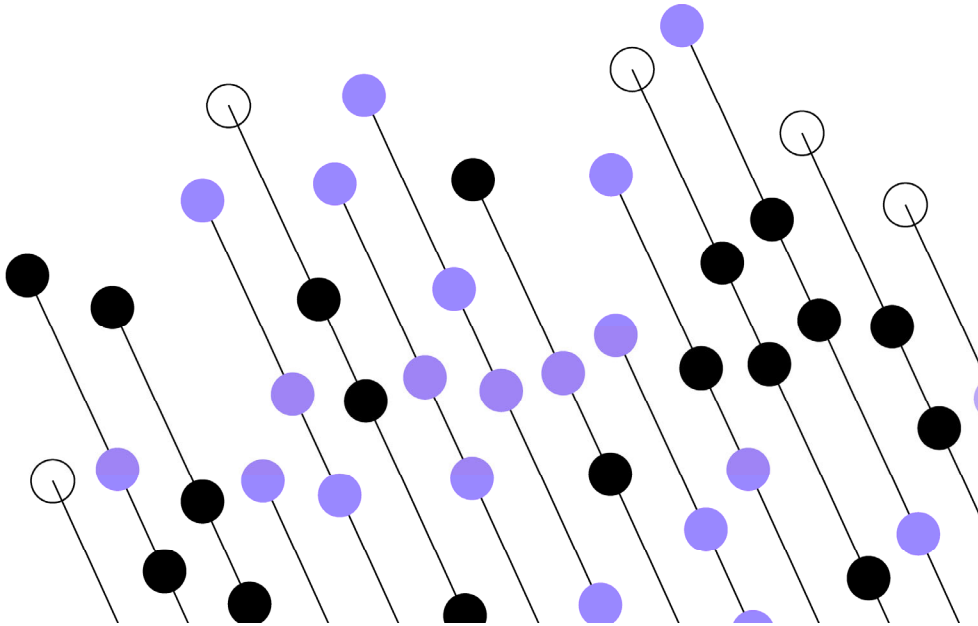
Exceptions



Contents

Objectives

- To explain exception handling in Java and C#.

Contents

- Exception handling syntax.
- Throwing exceptions.
- Understanding execution flow with exceptions.
- The try-with-resources statement.

Hands-on labs

Example of exception being thrown by a coding error

```
using System;

class User {
    private String username;

    public User(String username) {
        this.username = username;
    }

    public String getName() {
        return username;
    }
}
```

```
public class Program {
    public static void main(String[] args) {
        User user = new User(null);
        String username = user.getName();
        System.out.println(username.length());
    }
}
```

```
Exception in thread "main" java.lang.NullPointerException:
Cannot invoke "String.length()" because "username" is null
```

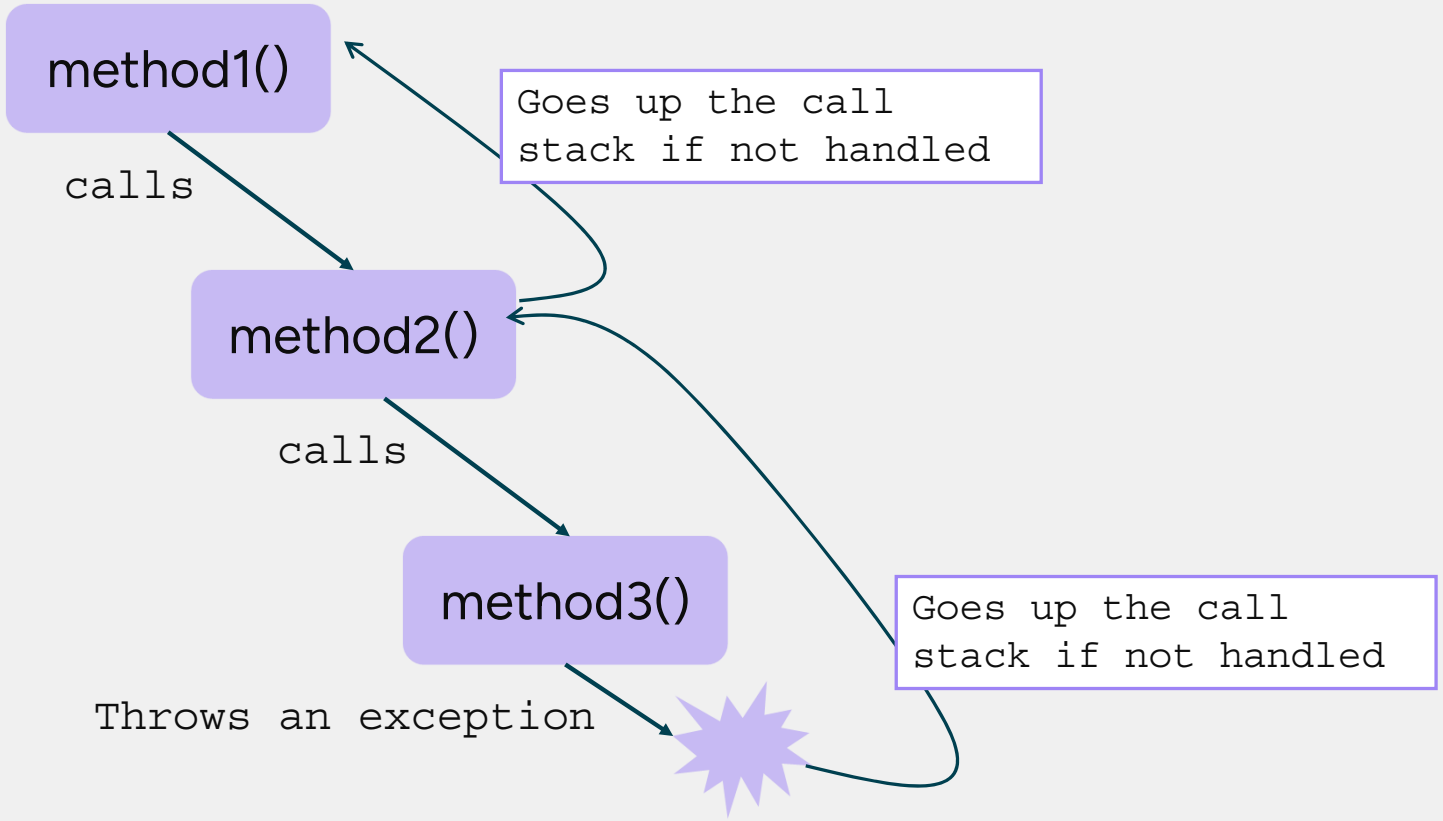
A few unpredictable exceptions

- **Accessing a file:**
  - AccessDeniedException
  - FileNotFoundException
- **Accessing objects:**
  - NullReferenceException
- **Networking:**
  - SSLException
  - ConnectException
  - SocketTimeoutException

Types of exceptions

- **Throwable is the super class of all exceptions.**
  - **Error:**
    - Typically, unrecoverable external error
      - Out of memory, stack overflow, etc.
    - Unchecked by compiler
  - **RuntimeException:**
    - Typically programming error
      - Unchecked by compiler
  - **Exception:**
    - Recoverable error (database / file IO, etc.)
    - **Checked** by Java compiler (must be caught or thrown)

Exceptions bubble up the call stack

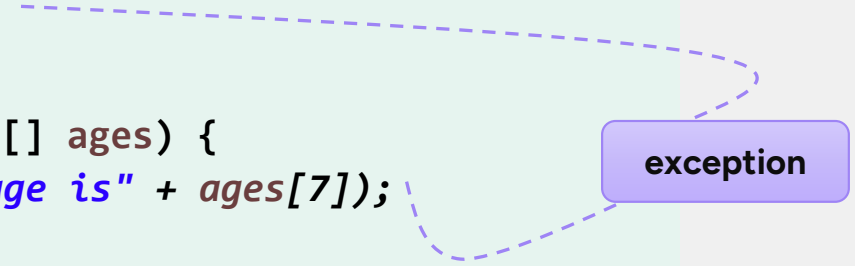


Example of exception bubbling up

- Same coding error but exception thrown in called method:

```
public static void main(String[] args) {
        int[] ages = new int[7];
        processAges(ages);
}

public static void processAges(int[] ages) {
        System.out.println("Last age is" + ages[7]);
}
```




```
Exception in thread "main"
ArrayIndexOutOfBoundsException: 7
```

try / catch / finally syntax

```
try {
  // guarded block
  execute_code();
}
catch( SomeSpecificExceptionType exn) {
  System.out.println("....: " + exn.getMessage());

}
catch( Exception exn ) {  // catch everything else
  System.out.println("General error: " + exn.getMessage());

}
finally {
  // closing files/connections
}
//remainder of containing method
```

Clean up and/or abort

Execute this whatever happens

Multiple exceptions example

Before Java-7

```
try {
        // some code
} catch (IOException ex) {
        logger.error(ex);
        throw new Exception("Cannot open file");
} catch (SQLException ex) {
        logger.error(ex);
        throw new Exception("Database error!");
}
```

Inform the caller

Java-7

```
try {
        // some code
} catch(IOException | SQLException ex){
        logger.error(ex);
        throw new Exception("Data access error!");
}
```

Java: Method throwing an exception ‘throws’

- **If a method contains a statement that throws a checked exception and then calls a method that throws an unhandled checked exception...**
- **...then it must ‘declare itself’ as throwing a checked exception.**
  - This enables the compiler to see that a catch clause is needed.
- **Let's see a code example...**

Method throwing checked exceptions

```
public static void main(String[] args) {
        try {
                readFile();
        } catch (FileNotFoundException e) {
                System.out.println(e.getMessage());
        } catch (IOException e) {
                System.out.println(e.getMessage());
        }
}
```

```
private static void readFile() throws IOException {
        File file = new File("fiction.txt");
        BufferedReader br = new BufferedReader(new FileReader(file));

        String st;
        while ((st = br.readLine()) != null) {
                System.out.println(st);
        }
}
```

throws **FileNotFoundException**

throws **IOException**

readFile() should either catch or throw the exceptions

Method throwing exceptions

```
public class Bank {
    public static void main(String[] args) {
        Account acc = new Account(0);
        // more code...
    }
}
```

Exception in thread "main" **java.lang.IllegalArgumentException**: ID less than 1

```
class Account {
    int id;
    public Account(int id) {
        if(id < 1)
                throw new IllegalArgumentException("ID less than 1");

        this.id = id;
    }
    // more code...
}
```

SE 7: Try-with-resource

- **A try-statement that releases resources on termination:**

fis will be closed() Even when an exception is thrown

```
try (FileInputStream fis = new FileInputStream(file)) {
    ...
}
catch (Exception ex) {
    ...
}
```

- Resource must implement the **`java.lang.AutoCloseable`** interface
  - **`close()`** method releases the resource

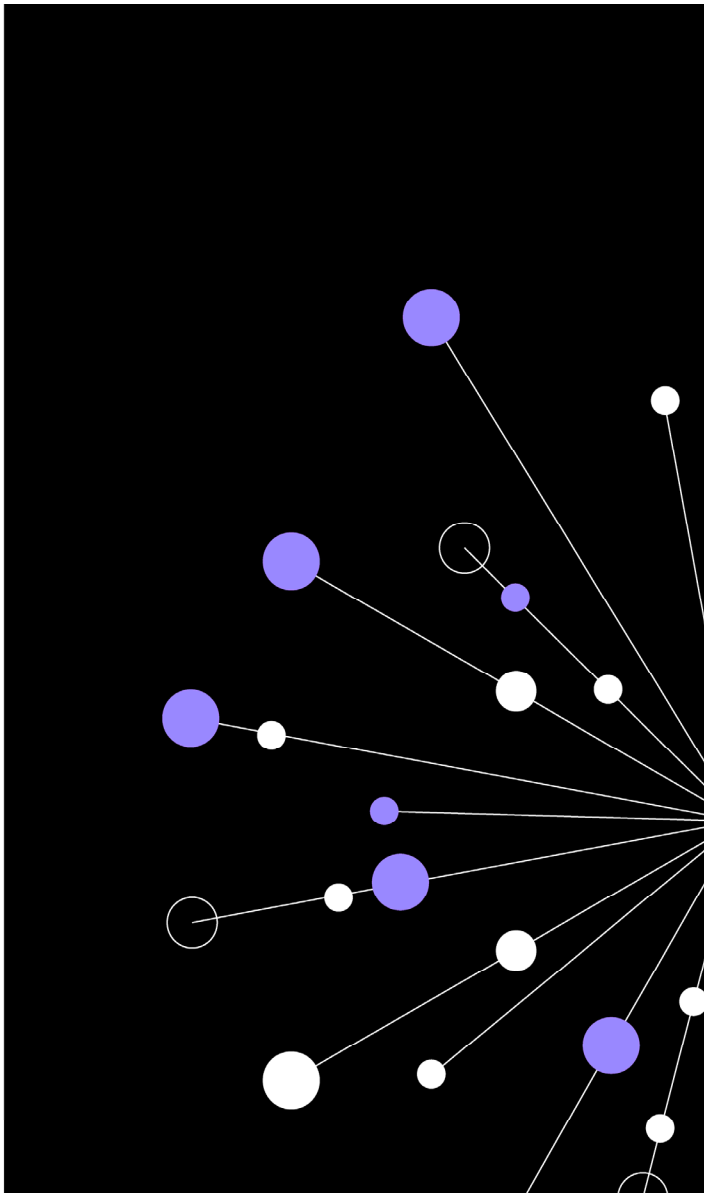
Best practice guidance

- **Don't only catch class throwable or exception.**
- **Don't try to catch every possible exception type.**
- **Have a ‘final’ catch of exception.**
- **You don't need try { } catch { } in every method.**
- **Use finally blocks to ensure resources are freed.**
- **Only use exceptions for exceptional circumstances.**
- **Be wary of just reporting back large error message.**
  - Might contain system information
  - Might be better with a user-friendly message

Review

- **Java uses exceptions to report errors.**
  - Use `try / catch / finally` blocks to encapsulate such code.
  - You can throw (or re-throw caught) exceptions.

- **Unhandled exceptions will be caught by the Java runtime.**

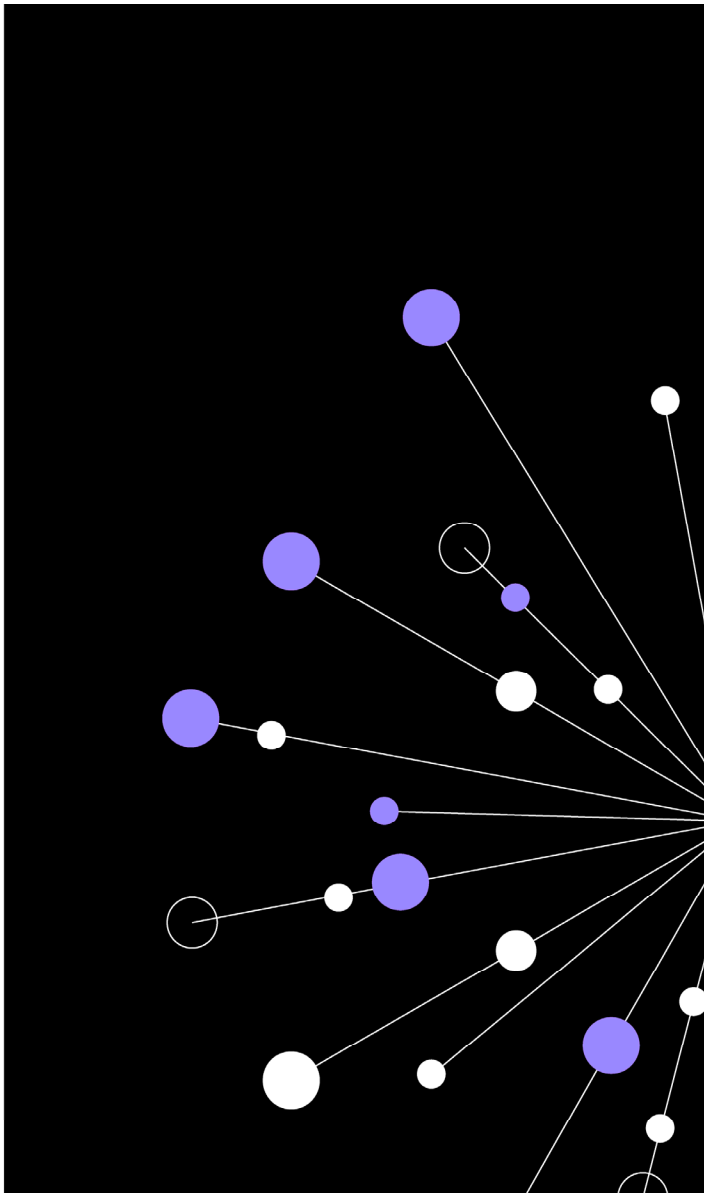


QA

Lab

- Working with exceptions

Duration: 60 minutes



QA